

Android瀑布流项目实践指南（Java版）

新手友好·步骤详细·覆盖所有要求

一、项目整体说明

1. 实现目标

- ✓ 瀑布流界面（StaggeredGridLayoutManager）
- ✓ 下拉刷新（SmartRefreshLayout）
- ✓ 上滑加载更多（子线程处理）
- ✓ Retrofit获取数据
- ✓ Glide加载图片（含圆角后处理）
- ✓ 图片地址：`https://picsum.photos/400/{height}`（height取200~800随机值）

2. 项目构思与运行流程

本项目采用**分层架构**设计，遵循**单一职责原则**，将UI展示、数据处理、网络请求等功能清晰分离。整个应用的构思和运行流程如下：

构思理念：

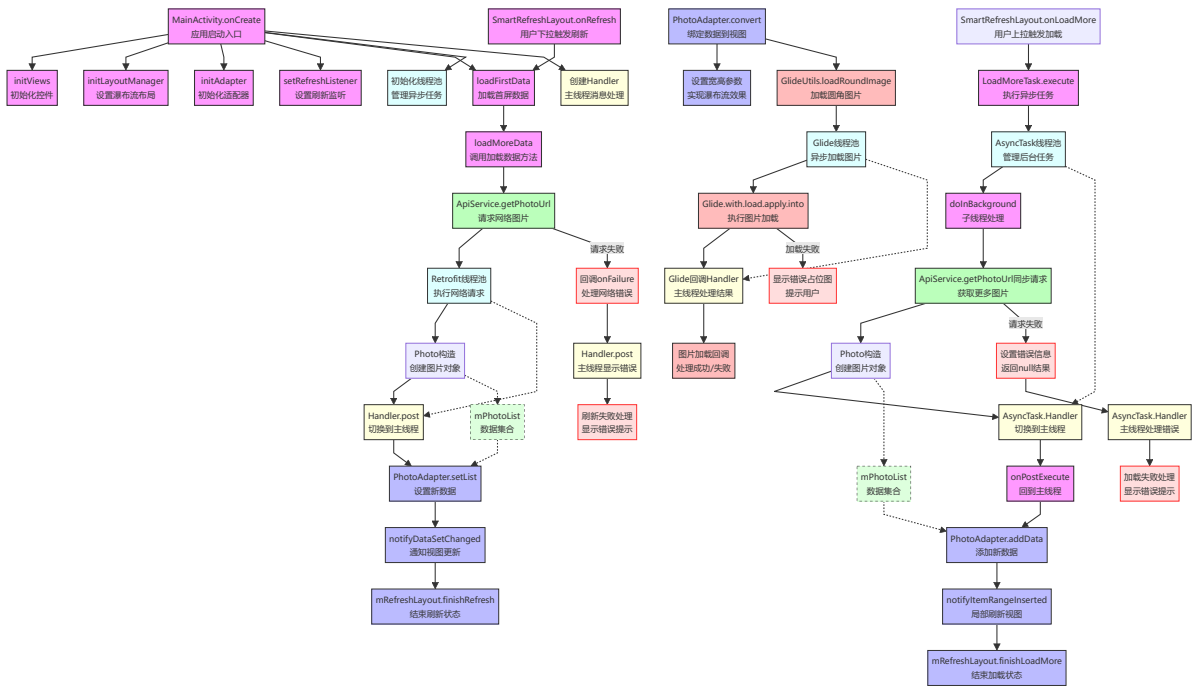
- 用户体验优先：**采用瀑布流布局展示不同高度的图片，视觉效果更加丰富；实现下拉刷新和上拉加载，交互体验流畅自然。
- 性能优化：**使用子线程处理耗时操作，避免主线程阻塞；适配器局部更新而非全量刷新，提高列表滚动流畅度。
- 代码复用：**封装网络请求、图片加载等通用功能，提高代码复用性和可维护性。
- 组件化思想：**选用成熟的第三方组件（SmartRefreshLayout、Retrofit、Glide、BRVAH），避免重复造轮子。

运行流程：

- 初始化阶段：**应用启动后，MainActivity初始化各个组件，包括视图控件、布局管理器、适配器和网络接口。
- 首屏加载：**通过Retrofit异步请求网络图片数据，获取随机高度的图片URL，创建Photo对象，通过适配器展示在瀑布流布局中。
- 下拉刷新：**用户下拉触发SmartRefreshLayout的刷新事件，清空现有数据，重新请求首屏数据，更新适配器并刷新视图。
- 上拉加载：**用户上拉到底部触发加载更多事件，启动AsyncTask在子线程中请求新数据，获取成功后添加到现有数据集合，局部更新视图。
- 图片展示：**适配器将数据绑定到视图时，设置不同的高度参数实现瀑布流效果，通过GlideUtils加载网络图片并应用圆角处理。

整个应用的核心在于**数据流转与视图更新**的协调，从网络请求到数据处理再到UI展示，形成完整的数据流链路。各组件间通过接口回调、事件监听等机制实现松耦合通信，使得代码结构清晰、易于维护和扩展。

3. 项目流程图（优化版）



4. 完整项目目录结构

```
app/
├─ src/
│  ├─ main/
│  │  ├─ java/
│  │  │  └─ com.example.waterfallapp/ # 包名（可自定义）
│  │  │     ├─ model/ # 数据模型
│  │  │     │  └─ Photo.java # 图片数据类（url+height）
│  │  │     ├─ adapter/ # 适配器
│  │  │     │  └─ PhotoAdapter.java # RecyclerView适配器
│  │  │     ├─ network/ # 网络层
│  │  │     │  ├─ ApiService.java # 接口定义
│  │  │     │  └─ RetrofitClient.java # Retrofit单例
│  │  │     ├─ ui/ # 界面
│  │  │     │  └─ MainActivity.java # 主界面
│  │  │     └─ utils/ # 工具类
│  │  │        └─ GlideUtils.java # Glide图片处理工具
│  │  └─ res/
│  │     ├─ layout/ # 布局文件
│  │     │  ├─ activity_main.xml # 主布局（SmartRefresh+RecyclerView）
│  │     │  └─ item_photo.xml # 瀑布流item（ImageView）
│  │     ├─ drawable/ # 图片资源
│  │     │  ├─ placeholder.png # 图片占位图（可选）
│  │     │  └─ error.png # 图片加载错误图（可选）
│  │     └─ values/ # 配置文件
│  │         ├─ strings.xml # 字符串
│  │         └─ dims.xml # 尺寸
│  └─ AndroidManifest.xml # 清单文件（添加网络权限）
├─ build.gradle # 模块级依赖配置
└─ build.gradle # 项目级gradle配置
```

二、基础知识铺垫（必看）

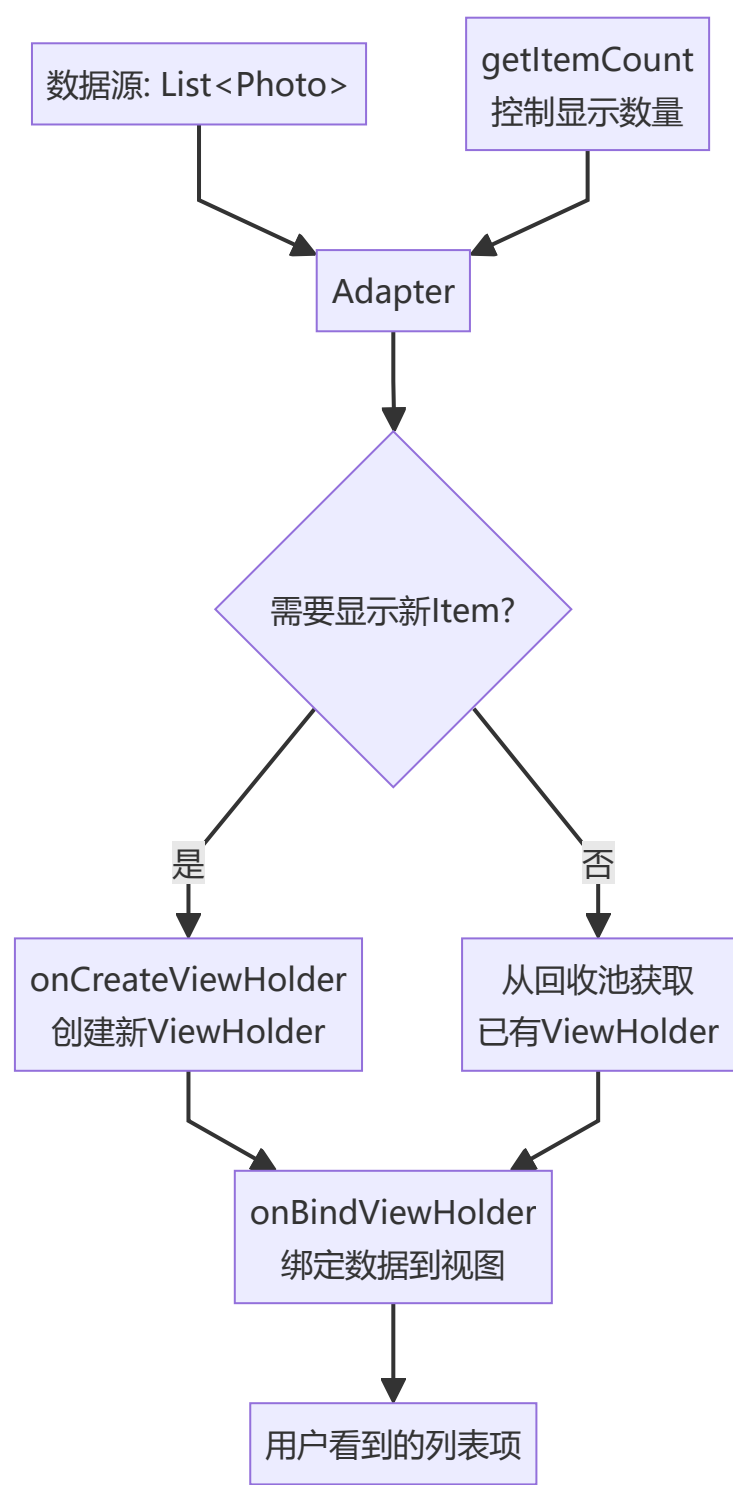
1. RecyclerView Adapter 核心逻辑

1.1 组件角色与职责

RecyclerView的 Adapter 是数据与视图之间的桥梁，可以用「工厂生产产品」类比：

组件	工厂比喻	职责
RecyclerView.Adapter	工厂总控	管理数据集合，协调视图创建和数据绑定流程
ViewHolder	产品模具	持有item视图引用，避免重复findViewById
onCreateViewHolder()	造盒子	加载布局文件，创建ViewHolder实例
onBindViewHolder()	填充内容	将数据绑定到ViewHolder持有的视图上
getItemCount()	计数器	告知RecyclerView需要显示多少个item

1.2 工作流程图



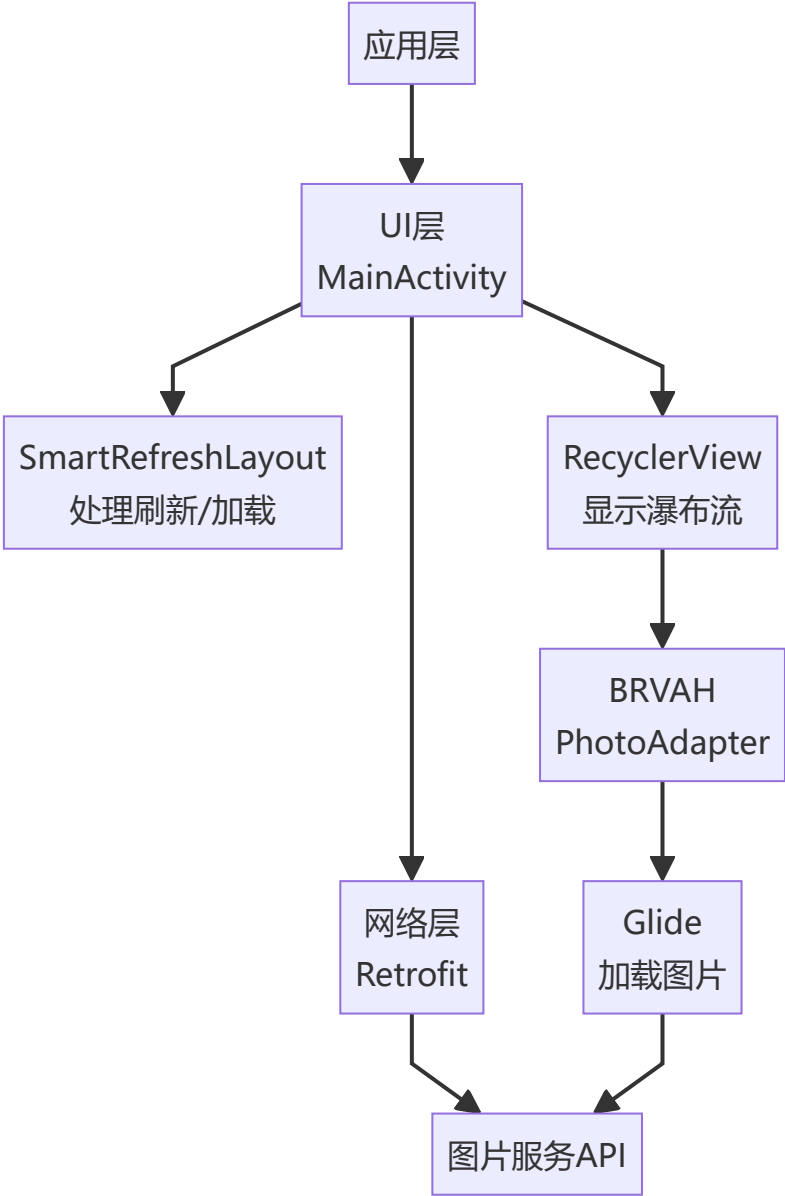
2. 关键依赖说明

2.1 核心库及其功能

依赖库	主要功能	优势特点	在项目中的作用
SmartRefreshLayout	下拉刷新/上拉加载	丰富的动画效果、高度可定制	提供瀑布流的刷新和加载更多功能
Retrofit	网络请求	类型安全、注解驱动、易扩展	获取随机图片URL

依赖库	主要功能	优势特点	在项目中的作用
Glide	图片加载	高效缓存、内存管理、生命周期感知	加载网络图片并应用圆角效果
BRVAH	简化适配器开发	减少模板代码、提供丰富功能	实现瀑布流适配器，简化开发

2.2 依赖关系图



3. BaseRecyclerViewAdapterHelper优势

3.1 BRVAH vs 原生适配器对比

功能特性	原生 RecyclerView.Adapter	BRVAH	开发效率提升
ViewHolder创建	需手动创建内部类	自动创建，无需额外代码	↑↑↑

功能特性	原生 RecyclerView.Adapter	BRVAH	开发效率提升
控件绑定	需使用findViewById	提供getView方法简化操作	↑ ↑
数据操作	需自行实现添加/删除方法	内置addData/setList等方法	↑ ↑ ↑
多布局支持	需复杂判断逻辑	简单配置即可实现	↑ ↑ ↑
动画效果	需自行实现	内置多种动画效果	↑ ↑
事件处理	需自行实现	内置点击/长按事件	↑ ↑
加载状态	需自行实现	内置加载更多、空布局等	↑ ↑ ↑

3.2 代码对比示例

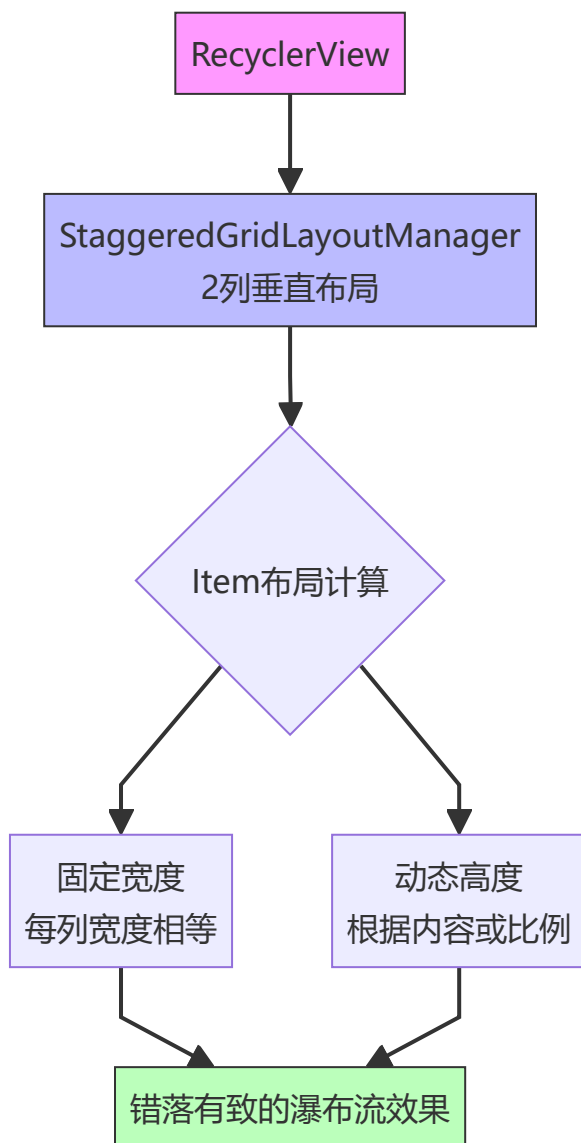


4. 瀑布流布局特点与实现

4.1 瀑布流布局特点

特点	描述	优势
视觉丰富度	不同高度内容错落有致	符合现代审美，避免单调感
空间利用率	根据内容自适应高度	充分利用屏幕空间，减少空白
用户体验	浏览更自然流畅	类似Pinterest等流行应用的体验
内容展示	强调内容本身	突出图片等视觉内容的表现力

4.2 实现原理图解



4.3 核心实现代码

```
// 1. 创建瀑布布局管理器（2列，垂直方向）
StaggeredGridLayoutManager layoutManager = new StaggeredGridLayoutManager(
    2, StaggeredGridLayoutManager.VERTICAL
);
recyclerView.setLayoutManager(layoutManager);

// 2. 在适配器中设置不同高度（关键点）
@Override
protected void convert(@NonNull BaseViewHolder holder, Photo photo) {
    ImageView imageView = holder.getView(R.id.iv_photo);

    // 设置固定宽度、动态高度
    ViewGroup.LayoutParams params = imageView.getLayoutParams();
    params.width = mItemWidth; // 固定宽度（屏幕宽/列数）
    params.height = photo.getHeight(); // 动态高度
    imageView.setLayoutParams(params);

    // 加载图片
    GlideUtils.loadRoundImage(getContext(), photo.getUrl(), imageView);
}
```

```
}

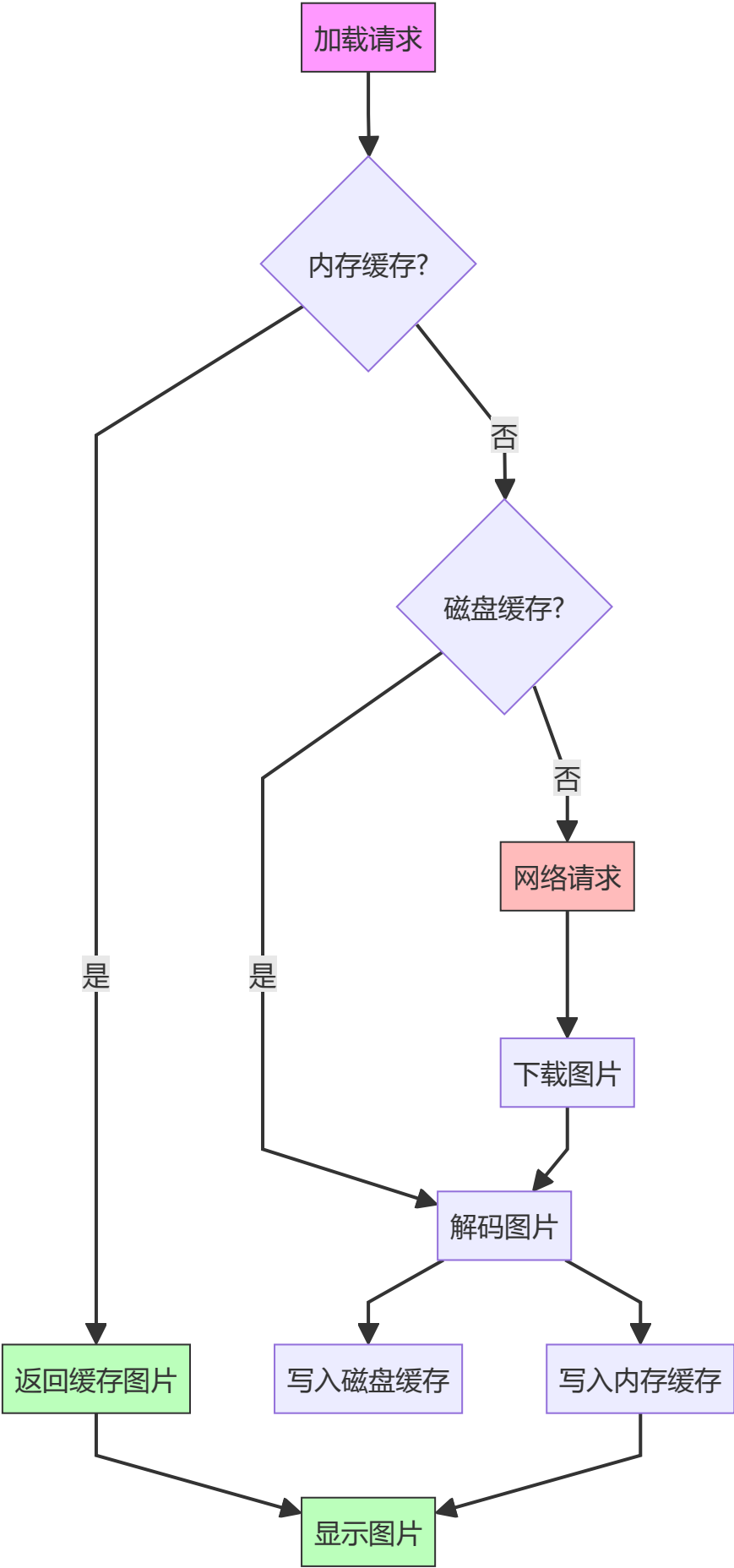
```

5. Glide图片加载机制

5.1 Glide核心特性

特性	描述	优势
多级缓存	内存缓存 + 磁盘缓存	减少网络请求，提高加载速度
生命周期管理	与Activity/Fragment生命周期绑定	避免内存泄漏，自动管理资源
图片转换	裁剪、圆角、模糊等效果	满足多样化的UI需求
资源池管理	Bitmap复用池	减少内存抖动，避免频繁GC
高效解码	渐进式图片解码	优化大图加载体验

5.2 Glide工作流程图



5.3 Glide在项目中的应用

```
// 封装Glide加载圆角图片
public static void loadRoundImage(Context context, String url, ImageView
imageView) {
    RequestOptions options = new RequestOptions()
        // 1. 应用圆角变换
        .transform(new RoundedCorners(dp2px(context, 16)))
        // 2. 设置占位图和错误图
        .placeholder(R.drawable.placeholder)
        .error(R.drawable.error)
        // 3. 缩放模式
        .centerCrop();

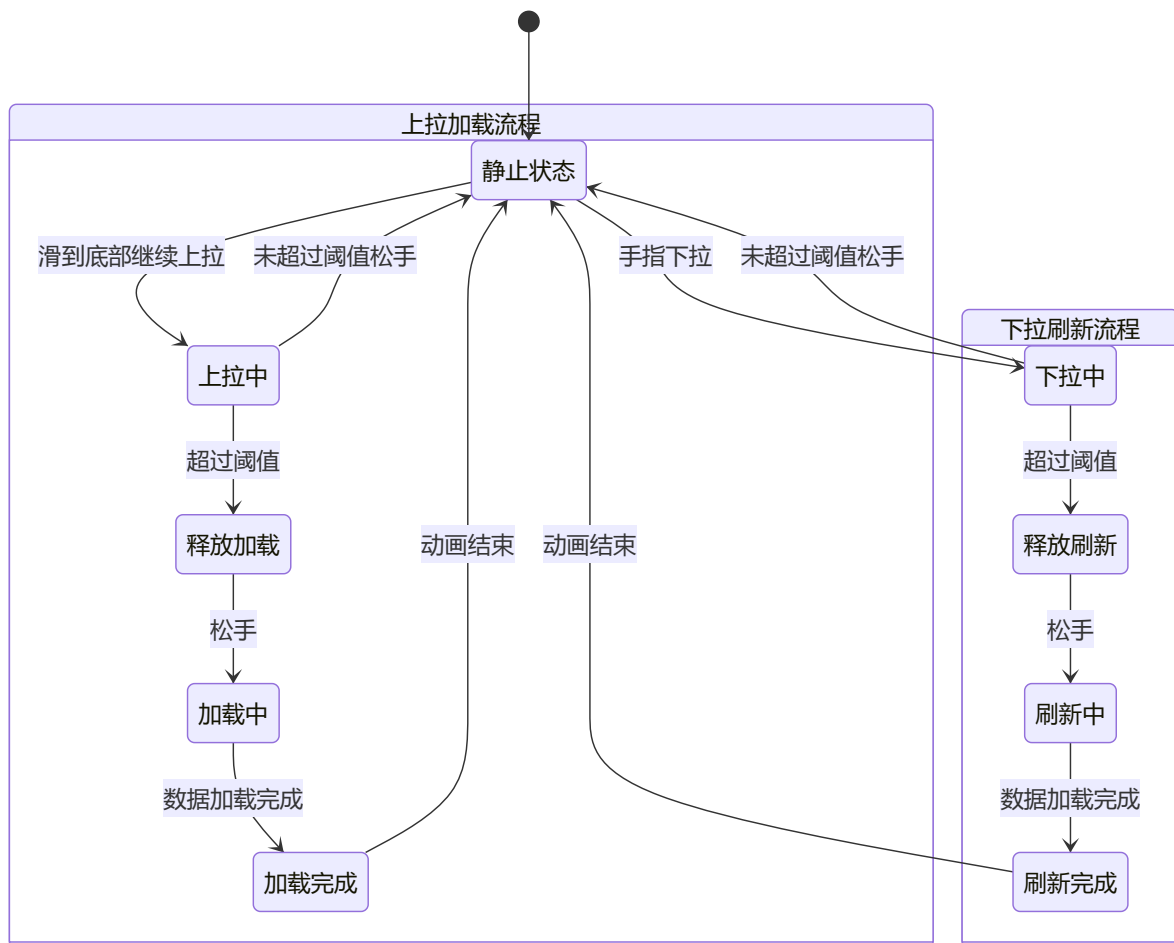
    // 4. 执行加载
    Glide.with(context) // 生命周期绑定
        .load(url)       // 加载源
        .apply(options)  // 应用选项
        .into(imageView); // 目标视图
}
```

6. 下拉刷新与上拉加载机制

6.1 交互流程对比

步骤	下拉刷新	上拉加载
触发方式	用户手指下拉屏幕顶部	用户滑动到列表底部
视觉反馈	显示刷新状态指示器	显示加载更多指示器
事件回调	onRefresh()	onLoadMore()
数据处理	清空旧数据，加载首页数据	保留旧数据，追加新数据
UI更新	setList() 替换数据	addData() 追加数据
完成状态	finishRefresh()	finishLoadMore()

6.2 SmartRefreshLayout工作流程图



6.3 SmartRefreshLayout实现代码

```
// 设置下拉刷新和上拉加载监听
private void setRefreshListener() {
    // 下拉刷新监听
    mRefreshLayout.setOnRefreshListener(refreshLayout -> {
        // 清空旧数据，加载首页数据
        loadData(true);
    });

    // 上拉加载更多监听
    mRefreshLayout.setOnLoadMoreListener(refreshLayout -> {
        // 保留旧数据，加载更多数据
        loadData(false);
    });
}

// 处理刷新和加载结果
private void handleResult(List<Photo> photos, boolean isRefresh, boolean success) {
    if (isRefresh) {
        // 下拉刷新结果处理
        if (success) {
            mAdapter.setList(photos); // 替换数据
            mRefreshLayout.finishRefresh(true); // 完成刷新（成功）
        } else {

```

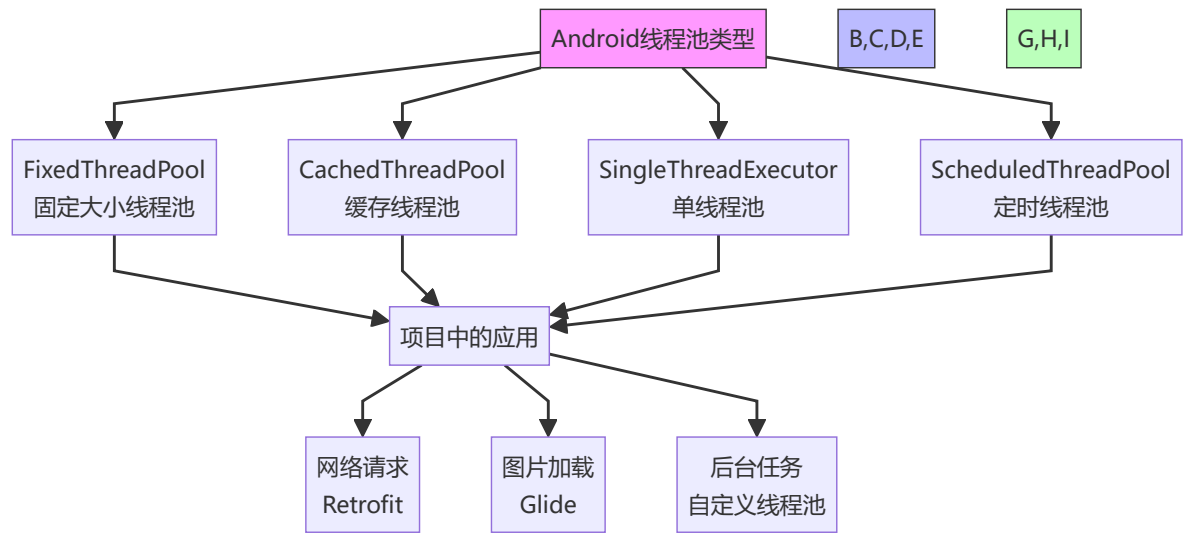
```
mRefreshLayout.finishRefresh(false); // 完成刷新（失败）
    }
} else {
    // 上拉加载结果处理
    if (success) {
        if (photos.size() > 0) {
            mAdapter.addData(photos); // 追加数据
            mRefreshLayout.finishLoadMore(true); // 完成加载（成功）
        } else {
            mRefreshLayout.finishLoadMorewithNoMoreData(); // 没有更多数据
        }
    } else {
        mRefreshLayout.finishLoadMore(false); // 完成加载（失败）
    }
}
}
```

7. 线程池与Handler机制

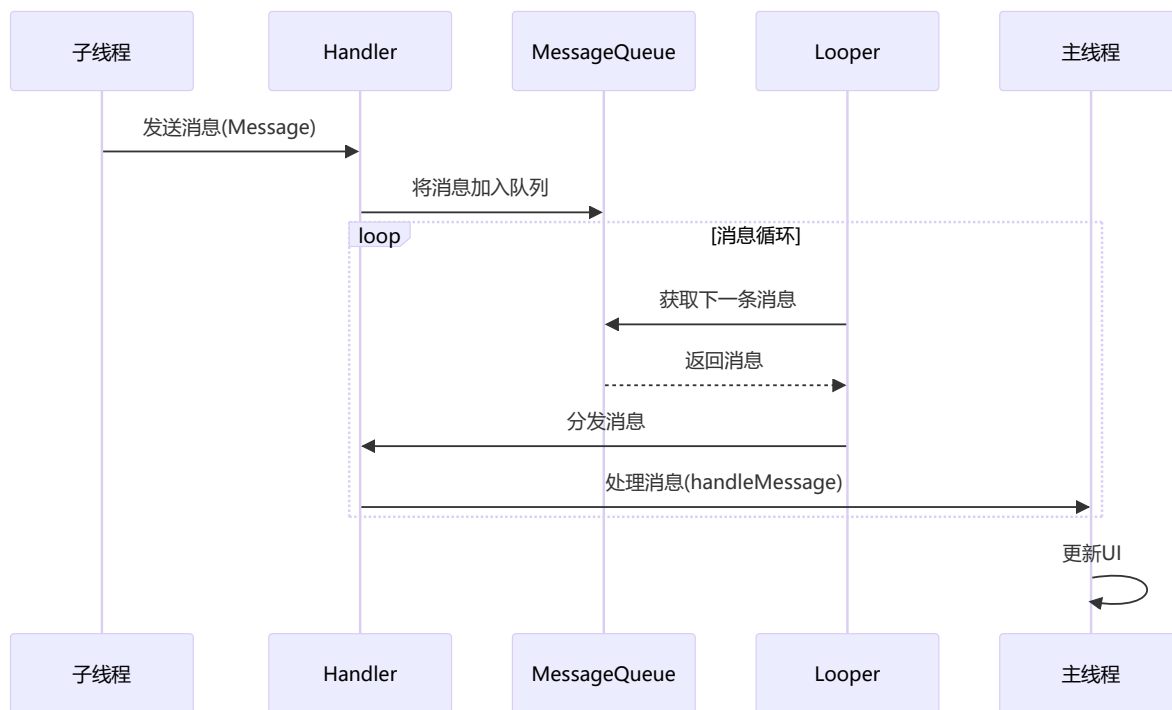
7.1 线程池与Handler对比表

特性	线程池(ThreadPool)	Handler机制
主要功能	管理和复用线程	线程间通信
使用场景	并发任务执行	子线程向主线程传递结果
核心优势	资源管理、性能优化	线程安全的UI更新
项目应用	网络请求、图片加载	异步结果回调主线程

7.2 线程池分类与应用



7.3 Handler工作原理图



7.4 项目中的线程池与Handler实现

```
// 1. 创建线程池
private ExecutorService mThreadPool = Executors.newFixedThreadPool(3);

// 2. 创建Handler(主线程)
private Handler mMainHandler = new Handler(Looper.getMainLooper());

// 3. 使用线程池执行异步任务
private void loadData(boolean isRefresh) {
    // 提交任务到线程池
    mThreadPool.execute(() -> {
        try {
            // 子线程中执行耗时操作
            List<Photo> photos = loadPhotosFromNetwork();

            // 通过Handler将结果传回主线程
            mMainHandler.post(() -> {
                // 在主线程中安全地更新UI
                if (isRefresh) {
                    mAdapter.setList(photos);
                    mRefreshLayout.finishRefresh(true);
                } else {
                    mAdapter.addData(photos);
                    mRefreshLayout.finishLoadMore(true);
                }
            });
        } catch (Exception e) {
            // 错误处理也需要回到主线程
            mMainHandler.post(() -> {
                // 在主线程中显示错误
                Toast.makeText(context, "加载失败", Toast.LENGTH_SHORT).show();
            });
        }
    });
}
```

```
        if (isRefresh) {
            mRefreshLayout.finishRefresh(false);
        } else {
            mRefreshLayout.finishLoadMore(false);
        }
    });
}
});
}
```

三、具体实现步骤（从0到1）

步骤1：创建Android项目

目的： 搭建基础开发环境，确定项目架构和兼容性要求。

1.1 创建新项目

目的： 初始化Android项目，设置基本配置。

1. 打开Android Studio，选择「New Project」→「Empty Activity」。
2. 填写项目信息：
 - 项目名： `waterfallApp`
 - 包名： `com.example.waterfallapp`（可自定义）
 - 语言： `Java`
 - 最小SDK： `API 21`（兼容Android 5.0+）

项目环境说明

目的： 明确项目的构建环境和代码语言，避免配置混淆。

本项目使用**Kotlin DSL构建系统**（`build.gradle.kts`）进行项目配置，但**主要代码使用Java语言**编写。这种混合方式在Android开发中很常见，允许我们利用Kotlin DSL的优势进行构建配置，同时保持Java代码的兼容性。

步骤2：导入依赖（模块级`build.gradle.kts`）

目的： 引入必要的第三方库，避免重复造轮子，提高开发效率。

2.1 添加应用依赖

目的： 在模块级`build.gradle.kts`中添加项目所需的所有依赖库。

`app/build.gradle.kts`

```
// 1. 瀑布流+下拉刷新: SmartRefreshLayout
implementation("com.scwang.smart:refresh-layout-kernel:2.0.5")
implementation("com.scwang.smart:refresh-header-classics:2.0.5")
implementation("com.scwang.smart:refresh-footer-classics:2.0.5")

// 2. 网络请求: Retrofit+Gson
implementation("com.squareup.retrofit2:retrofit:2.9.0")
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

```
// 3. 图片加载: Glide
implementation("com.github.bumptech.glide:glide:4.16.0")
annotationProcessor("com.github.bumptech.glide:compiler:4.16.0")

// 4. BaseRecyclerViewAdapterHelper
implementation("com.github.CymChad:BaseRecyclerViewAdapterHelper:3.0.10")

// 5. 其他: RecyclerView (AndroidX)
implementation("androidx.recyclerview:recyclerview:1.3.2")
```

注意：添加后点击「Sync Now」同步依赖。

2.2 添加JitPack仓库

目的：添加jitPack仓库，以便能够下载BaseRecyclerViewAdapterHelper库。

settings.gradle.kts

```
dependencyResolutionManagement {
    repositories {
        google()
        mavenCentral()
        maven(url = "https://jitpack.io") // 添加JitPack仓库
    }
}
```

2.3 Java兼容性配置

目的：确保Kotlin构建系统正确处理Java代码。

在模块级 build.gradle.kts 中添加Java兼容性配置：

```
android {
    // 其他配置...

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
}
```

步骤3：添加网络权限 (AndroidManifest.xml)

目的：确保应用有访问网络的权限，否则无法获取网络图片。

3.1 修改AndroidManifest.xml

目的：添加网络权限声明，允许应用访问互联网。

app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.waterfallapp">
```

```

<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.WaterfallApp">
    <activity
        android:name=".ui.MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
</manifest>

```

新增步骤：Kotlin环境配置

目的： 确保项目正确配置为使用Kotlin语言开发。

Kotlin配置检查

目的： 验证项目已正确配置为使用Kotlin。

1. 确认项目级 `build.gradle.kts` 中已添加Kotlin插件：

```

plugins {
    id("org.jetbrains.kotlin.android") version "1.9.0" apply false
}

```

2. 确认模块级 `build.gradle.kts` 中已应用Kotlin插件并配置编译选项：

```

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    // 其他配置...

    kotlinOptions {
        jvmTarget = "17" // 确保与Java兼容性
        freeCompilerArgs += listOf("-Xjvm-default=all") // 启用默认接口方法
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
}

```

3. 确认项目结构中的源代码目录正确设置：

```
app/  
├─ src/  
│   └─ main/  
│       └─ java/ (可能包含Kotlin文件)  
│       └─ kotlin/ (可能的额外Kotlin目录)
```

注意：现代Android Studio项目通常将Kotlin文件放在 `java` 目录中，但也可能有专门的 `kotlin` 目录。

步骤4：编写布局文件

目的： 创建UI界面结构，实现瀑布流布局 and 下拉刷新容器。

4.1 主布局： `activity_main.xml`

目的： 创建主界面布局，设置下拉刷新容器和瀑布流列表。

`app/src/main/res/layout/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>  
<com.scwang.smart.refresh.layout.SmartRefreshLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    android:id="@+id/refresh_layout"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
  
    <androidx.recyclerview.widget.RecyclerView  
        android:id="@+id/recycler_view"  
        android:layout_width="match_parent"  
        android:layout_height="match_parent"  
        android:clipToPadding="false"  
        android:padding="8dp" />  
  
</com.scwang.smart.refresh.layout.SmartRefreshLayout>
```

说明：

- `SmartRefreshLayout`：包裹`RecyclerView`，实现下拉刷新和上滑加载。
- `RecyclerView`：在代码中设置为2列瀑布流，`clipToPadding="false"` + `padding="8dp"` 让item之间有间距。

4.2 Item布局： `item_photo.xml`

目的： 创建瀑布流中每个图片项的布局。

`app/src/main/res/layout/item_photo.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<ImageView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/iv_photo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:scaleType="centerCrop"
    android:margin="4dp" />
```

步骤5：定义数据模型

目的： 创建数据实体类，封装图片信息（URL和高度）。

5.1 创建Photo类

目的： 定义图片数据模型，存储URL和高度信息。

app/src/main/java/com/example/waterfallapp/model/Photo.java

```
package com.example.waterfallapp.model;

public class Photo {
    private String url; // 图片地址
    private int height; // 图片高度（200~800）

    public Photo(String url, int height) {
        this.url = url;
        this.height = height;
    }

    public String getUrl() {
        return url;
    }

    public int getHeight() {
        return height;
    }
}
```

步骤6：实现网络请求

目的： 构建网络层，实现与服务器的通信，获取图片数据。

6.1 定义接口：ApiService

目的： 定义网络请求接口，指定API路径和参数。

app/src/main/java/com/example/waterfallapp/network/ApiService.java

```
package com.example.waterfallapp.network;

import retrofit2.Call;
import retrofit2.http.GET;
import retrofit2.http.Path;
```

```

public interface ApiService {
    /**
     * 获取图片地址
     * 示例: https://picsum.photos/400/300
     * @param width 图片宽度 (固定400)
     * @param height 图片高度 (随机200~800)
     */
    @GET("{width}/{height}")
    Call<Void> getPhotoUrl(
        @Path("width") int width,
        @Path("height") int height
    );
}

```

6.2 Retrofit单例: RetrofitClient

目的: 创建Retrofit单例, 配置网络请求客户端, 避免重复创建实例。

app/src/main/java/com/example/waterfallapp/network/RetrofitClient.java

```

package com.example.waterfallapp.network;

import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class RetrofitClient {
    // 基础URL
    private static final String BASE_URL = "https://picsum.photos/";
    // 单例对象
    private static volatile Retrofit retrofit;

    // 私有构造方法 (防止外部实例化)
    private RetrofitClient() {}

    // 获取Retrofit实例
    public static Retrofit getInstance() {
        if (retrofit == null) {
            synchronized (RetrofitClient.class) {
                if (retrofit == null) {
                    retrofit = new Retrofit.Builder()
                        .baseUrl(BASE_URL)
                        .addConverterFactory(GsonConverterFactory.create())
                        .build();
                }
            }
        }
        return retrofit;
    }

    // 获取ApiService实例
    public static ApiService getApiService() {
        return getInstance().create(ApiService.class);
    }
}

```

步骤7：实现瀑布流适配器

目的： 使用BaseRecyclerViewAdapterHelper库创建适配器，实现瀑布流布局效果。

7.1 创建PhotoAdapter类

目的： 实现瀑布流适配器，处理图片数据与视图的绑定。

app/src/main/java/com/example/waterfallapp/adapter/PhotoAdapter.java

```
package com.example.waterfallapp.adapter;

import android.view.ViewGroup;
import android.widget.ImageView;
import androidx.annotation.NonNull;
import com.chad.library.adapter.base.BaseQuickAdapter;
import com.chad.library.adapter.base.viewholder.BaseViewHolder;
import com.example.waterfallapp.R;
import com.example.waterfallapp.model.Photo;
import com.example.waterfallapp.utils.GlideUtils;

public class PhotoAdapter extends BaseQuickAdapter<Photo, BaseViewHolder> {
    // 每列的宽度（用于设置ImageView的宽度，保持瀑布流效果）
    private int mItemWidth;

    /**
     * 构造方法
     * @param itemwidth 每列宽度
     */
    public PhotoAdapter(int itemwidth) {
        super(R.layout.item_photo);
        this.mItemWidth = itemwidth;
    }

    @Override
    protected void convert(@NonNull BaseViewHolder holder, Photo photo) {
        // 获取ImageView控件
        ImageView ivPhoto = holder.getView(R.id.iv_photo);

        // 设置ImageView的宽度和高度（实现瀑布流效果）
        ViewGroup.LayoutParams params = ivPhoto.getLayoutParams();
        params.width = mItemWidth;
        params.height = photo.getHeight();
        ivPhoto.setLayoutParams(params);

        // 用Glide加载图片
        GlideUtils.loadRoundImage(getContext(), photo.getUrl(), ivPhoto);
    }
}
```

说明： BaseRecyclerViewAdapterHelper库简化了适配器的实现：

1. 无需手动创建ViewHolder类
2. 提供getView方法简化findViewByld操作
3. 内置常用数据操作方法（addData、setList等）

步骤8：实现图片加载工具类

目的： 封装图片加载逻辑，实现圆角效果，提高代码复用性。

8.1 创建GlideUtils工具类

目的： 封装Glide图片加载逻辑，统一处理圆角、占位图等配置。

app/src/main/java/com/example/waterfallapp/Utils/GlideUtils.java

```
package com.example.waterfallapp.Utils;

import android.content.Context;
import android.widget.ImageView;
import com.bumptech.glide.Glide;
import com.bumptech.glide.load.resource.bitmap.RoundedCorners;
import com.bumptech.glide.request.RequestOptions;
import com.example.waterfallapp.R;

public class GlideUtils {
    /**
     * 加载圆角图片
     */
    public static void loadRoundImage(Context context, String url, ImageView
imageView) {
        // 配置请求选项（圆角、占位图、错误图）
        RequestOptions options = new RequestOptions()
            .transform(new RoundedCorners(dp2px(context, 16))) // 16dp圆角
            .placeholder(R.drawable.placeholder) // 占位图
            .error(R.drawable.error) // 错误图
            .centerCrop(); // 缩放模式

        // 加载图片
        Glide.with(context)
            .load(url)
            .apply(options)
            .into(imageView);
    }

    /**
     * dp转px
     */
    private static int dp2px(Context context, int dp) {
        float density = context.getResources().getDisplayMetrics().density;
        return (int) (dp * density + 0.5f);
    }
}
```

步骤9：主界面逻辑

目的： 整合所有组件，实现核心功能（瀑布流、下拉刷新、上滑加载），处理用户交互。

9.1 创建MainActivity类

目的： 实现主界面，整合各个组件，处理用户交互。

app/src/main/java/com/example/waterfallapp/ui/MainActivity.java

```
package com.example.waterfallapp.ui;

import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.StaggeredGridLayoutManager;

import com.example.waterfallapp.R;
import com.example.waterfallapp.adapter.PhotoAdapter;
import com.example.waterfallapp.model.Photo;
import com.example.waterfallapp.network.ApiService;
import com.example.waterfallapp.network.RetrofitClient;
import com.scwang.smart.refresh.layout.SmartRefreshLayout;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import retrofit2.Response;

public class MainActivity extends AppCompatActivity {
    // 常量
    private static final int PAGE_SIZE = 10;

    // 成员变量
    private Random mRandom = new Random();
    private ApiService mApiService;
    private androidx.recyclerview.widget.RecyclerView mRecyclerView;
    private SmartRefreshLayout mRefreshLayout;
    private PhotoAdapter mAdapter;
    private Handler mHandler = new Handler(Looper.getMainLooper());
    private ExecutorService mThreadPool = Executors.newFixedThreadPool(3);

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // 初始化视图
        initView();

        // 设置瀑布流布局
        initLayoutManager();

        // 初始化适配器
```

```

initAdapter();

// 获取API服务
mApiService = RetrofitClient.getApiService();

// 设置刷新监听
setRefreshListener();

// 加载首屏数据
loadData(true);
}

// 初始化视图
private void initView() {
    mRefreshLayout = findViewById(R.id.refresh_layout);
    mRecyclerView = findViewById(R.id.recycler_view);
}

// 初始化布局管理器
private void initLayoutManager() {
    // 创建2列垂直瀑布流
    StaggeredGridLayoutManager layoutManager = new
StaggeredGridLayoutManager(
        2, StaggeredGridLayoutManager.VERTICAL);
    mRecyclerView.setLayoutManager(layoutManager);

    // 计算每列宽度
    int screenWidth = getResources().getDisplayMetrics().widthPixels;
    int itemWidth = (screenWidth - 16) / 2; // 考虑padding

    // 保存列宽
    mItemWidth = itemWidth;
}

// 初始化适配器
private void initAdapter() {
    mAdapter = new PhotoAdapter(mItemWidth);
    mRecyclerView.setAdapter(mAdapter);
}

// 设置刷新和加载监听
private void setRefreshListener() {
    // 下拉刷新
    mRefreshLayout.setOnRefreshListener(refreshLayout -> loadData(true));

    // 上拉加载更多
    mRefreshLayout.setOnLoadMoreListener(refreshLayout -> loadData(false));
}

// 加载数据（刷新或加载更多）
private void loadData(boolean isRefresh) {
    // 使用线程池异步加载
    mThreadPool.execute(() -> {
        try {
            List<Photo> photos = new ArrayList<>();

```

```

        // 生成随机高度的图片数据
        for (int i = 0; i < PAGE_SIZE; i++) {
            int height = mRandom.nextInt(601) + 200; // 200~800
            Response<Void> response = mApiService.getPhotoUrl(400,
height).execute();

            if (response.isSuccessful()) {
                String url = response.raw().request().url().toString();
                photos.add(new Photo(url, height));
            }
        }

        // 回到主线程更新UI
        mMainHandler.post(() -> {
            if (isRefresh) {
                mAdapter.setList(photos);
                mRefreshLayout.finishRefresh(true);
            } else {
                mAdapter.addData(photos);
                mRefreshLayout.finishLoadMore(true);
            }
        });
    } catch (Exception e) {
        // 处理错误
        mMainHandler.post(() -> {
            if (isRefresh) {
                mRefreshLayout.finishRefresh(false);
            } else {
                mRefreshLayout.finishLoadMore(false);
            }
            Toast.makeText(MainActivity.this, "加载失败: " +
e.getMessage(),
                                Toast.LENGTH_SHORT).show();
        });
    }
});
}

@Override
protected void onDestroy() {
    super.onDestroy();
    if (mThreadPool != null) {
        mThreadPool.shutdown();
    }
}
}

```

9.2 初始化视图和布局

目的： 初始化控件引用和设置瀑布流布局。

```

/**
 * 1. 初始化控件
 */
private void initView() {

```

```

        mRefreshLayout = findViewById(R.id.refresh_layout);
        mRecyclerView = findViewById(R.id.recycler_view);
    }

    /**
     * 2. 初始化瀑布布局管理器 (StaggeredGridLayoutManager)
     */
    private void initLayoutManager() {
        // 列数 (2列)
        int spanCount = 2;
        // 布局管理器 (垂直方向)
        StaggeredGridLayoutManager layoutManager = new
StaggeredGridLayoutManager(
            spanCount,
            StaggeredGridLayoutManager.VERTICAL
        );
        // 设置RecyclerView的布局管理器
        mRecyclerView.setLayoutManager(layoutManager);

        // 计算每列的宽度 (屏幕宽度/列数)
        int screenWidth = getResources().getDisplayMetrics().widthPixels;
        mItemWidth = screenWidth / spanCount;
    }

    /**
     * 3. 初始化适配器
     */
    private void initAdapter() {
        mAdapter = new PhotoAdapter(mPhotoList, mItemWidth);
        mRecyclerView.setAdapter(mAdapter);
    }

```

9.3 设置刷新和加载监听

目的： 配置下拉刷新和上滑加载的事件监听器。

```

    /**
     * 4. 设置下拉刷新和上滑加载监听
     */
    private void setRefreshListener() {
        // 下拉刷新监听
        mRefreshLayout.setOnRefreshListener(new OnRefreshListener() {
            @Override
            public void onRefresh(RefreshLayout refreshLayout) {
                // 刷新时, 重新加载首屏数据 (清空旧数据)
                loadFirstData();

                // 也可以设置刷新超时
                refreshLayout.finishRefresh(2000); // 2秒后自动结束刷新 (防止网络问题导致一
直刷新)
            }
        });

        // 上滑加载更多监听
        mRefreshLayout.setOnLoadMoreListener(new OnLoadMoreListener() {

```

```

@Override
public void onLoadMore(RefreshLayout refreshLayout) {
    // 加载更多时，用线程池在子线程处理
    loadMoreDataWithThreadPool(false);

    // 也可以设置加载超时
    refreshLayout.finishLoadMore(2000); // 2秒后自动结束加载（防止网络问题导致
一直加载）
}

});

// 添加刷新状态变化监听器（可选）
mRefreshLayout.setOnMultiListener(new SimpleMultiListener() {
    @Override
    public void onStateChanged(@NonNull RefreshLayout refreshLayout,
                                @NonNull RefreshState oldState,
                                @NonNull RefreshState newState) {
        // 状态变化监听，可以根据不同状态执行不同操作
        if (newState == RefreshState.Refreshing) {
            // 正在刷新
            Log.d(TAG, "正在刷新...");
        } else if (newState == RefreshState.Loading) {
            // 正在加载
            Log.d(TAG, "正在加载更多...");
        }
    }
});

@Override
public void onHeaderMoving(RefreshHeader header, boolean isDragging,
float percent,
                                int offset, int headerHeight, int
maxDragHeight) {
    // 下拉头部移动（可实现视差效果）
    // percent: 下拉的百分比 0.0f-2.0f
    // offset: 下拉的像素偏移量
}

});
}

```

9.4 加载首屏数据

目的： 实现首屏数据加载和下拉刷新功能。

```

/**
 * 5. 加载首屏数据（下拉刷新时也会调用）
 */
private void loadFirstData() {
    // 清空旧数据
    mPhotoList.clear();
    // 加载新数据（调用加载更多的方法，因为逻辑一致）
    loadMoreData(new LoadMoreCallback() {
        @Override
        public void onSuccess(List<Photo> morePhotos) {
            // 更新适配器（首屏用setList）
            mAdapter.setList(morePhotos); // BaseRecyclerViewAdapterHelper的方法
        }
    });
}

```

```

        // 停止下拉刷新
        mRefreshLayout.finishRefresh();
    }

    @Override
    public void onFailure(String errorMsg) {
        Log.e(TAG, "加载首屏失败: " + errorMsg);
        // 停止下拉刷新（带错误提示）
        mRefreshLayout.finishRefresh(false);
    }
}
});
}

```

9.5 通用数据加载方法

目的： 实现通用的数据加载逻辑，用于首屏和加载更多。

```

/**
 * 6. 加载更多数据（通用方法，首屏和加载更多都用）
 * @param callback 回调接口（处理成功/失败）
 */
private void loadMoreData(final LoadMoreCallback callback) {
    // 生成20个随机高度（200~800）
    List<Integer> heights = new ArrayList<>();
    for (int i = 0; i < PAGE_SIZE; i++) {
        heights.add(mRandom.nextInt(601) + 200); // 200~800（601=800-200+1）
    }

    // 批量请求图片地址（用Retrofit的enqueue异步请求）
    final List<Photo> morePhotos = new ArrayList<>();
    int count = 0; // 统计成功的数量

    for (final int height : heights) {
        mApiService.getPhotoUrl(0, 400, height, "").enqueue(new Callback<Void>() {
            @Override
            public void onResponse(Call<Void> call, Response<Void> response) {
                if (response.isSuccessful()) {
                    // 获取图片地址
                    String photoUrl = response.raw().request().url().toString();
                    // 添加到更多数据集合
                    morePhotos.add(new Photo(photoUrl, height));
                    count++;

                    // 当所有请求都成功时，回调成功
                    if (count == PAGE_SIZE) {
                        callback.onSuccess(morePhotos);
                    }
                } else {
                    // 请求失败，回调失败
                    callback.onFailure("请求图片地址失败: " +
response.message());
                }
            }
        });
    }
}

```

```

        @Override
        public void onFailure(Call<Void> call, Throwable t) {
            // 网络错误，回调失败
            callback.onFailure("网络错误: " + t.getMessage());
        }
    });
}

/**
 * 加载更多回调接口（用于处理成功/失败）
 */
private interface LoadMoreCallback {
    void onSuccess(List<Photo> morePhotos); // 成功回调
    void onFailure(String errorMsg); // 失败回调
}

```

9.6 异步加载任务

目的： 在子线程中处理上滑加载更多数据请求。

```

/**
 * 7. 上滑加载更多AsyncTask（子线程处理，满足+5分要求）
 * 泛型说明：Params（输入参数）、Progress（进度）、Result（输出结果）
 */
private class LoadMoreTask extends AsyncTask<Void, Void, List<Photo>> {
    private String errorMsg; // 错误信息

    @Override
    protected List<Photo> doInBackground(Void... voids) {
        // 子线程中执行：加载更多数据（同步请求，因为AsyncTask的doInBackground是子线程）

        final List<Photo> morePhotos = new ArrayList<>();
        final boolean[] isSuccess = {true}; // 用于标记是否成功（需要final，因为内部类访问）

        // 生成20个随机高度
        List<Integer> heights = new ArrayList<>();
        for (int i = 0; i < PAGE_SIZE; i++) {
            heights.add(mRandom.nextInt(601) + 200);
        }

        // 批量请求图片地址（同步请求，因为doInBackground是子线程）
        for (int height : heights) {
            try {
                // 同步请求（execute()）
                Response<Void> response = mApiService.getPhotoUrl(0, 400, height, "").execute();
                if (response.isSuccessful()) {
                    morePhotos.add(new Photo(response.raw().request().url().toString(), height));
                } else {
                    isSuccess[0] = false;
                    errorMsg = "请求图片地址失败: " + response.message();
                    break; // 停止循环
                }
            }

```

```

    }
    } catch (Exception e) {
        isSuccess[0] = false;
        mErrorMsg = "网络错误: " + e.getMessage();
        break; // 停止循环
    }
}

// 如果失败, 返回null
return isSuccess[0] ? morePhotos : null;
}

@Override
protected void onPostExecute(List<Photo> morePhotos) {
    // 主线程中执行: 处理结果
    if (morePhotos != null && !morePhotos.isEmpty()) {
        // 添加数据到适配器
        mAdapter.addData(morePhotos); // BaseRecyclerViewAdapterHelper的方法
        // 停止加载更多 (成功)
        mRefreshLayout.finishLoadMore();
    } else {
        Log.e(TAG, "加载更多失败: " + mErrorMsg);
        // 停止加载更多 (失败, 带错误提示)
        mRefreshLayout.finishLoadMore(false);
    }
}
}
}

```

9.7 添加Handler和线程池实现

目的: 优化异步操作, 确保线程安全和UI更新的正确性。

9.7.1 定义Handler处理UI更新

```

// 在MainActivity类中添加Handler成员变量
private Handler mMainHandler = new Handler(Looper.getMainLooper()) {
    @Override
    public void handleMessage(@NonNull Message msg) {
        switch (msg.what) {
            case MSG_LOAD_SUCCESS:
                // 处理加载成功
                List<Photo> photos = (List<Photo>) msg.obj;
                if (msg.arg1 == TYPE_REFRESH) {
                    // 下拉刷新成功
                    mAdapter.setList(photos);
                    mRefreshLayout.finishRefresh();
                } else {
                    // 加载更多成功
                    mAdapter.addData(photos);
                    mRefreshLayout.finishLoadMore();
                }
                break;

            case MSG_LOAD_FAILURE:
                // 处理加载失败

```

```

        String errorMsg = (String) msg.obj;
        if (msg.arg1 == TYPE_REFRESH) {
            // 下拉刷新失败
            mRefreshLayout.finishRefresh(false);
            Toast.makeText(MainActivity.this, "刷新失败: " + errorMsg,
Toast.LENGTH_SHORT).show();
        } else {
            // 加载更多失败
            mRefreshLayout.finishLoadMore(false);
            Toast.makeText(MainActivity.this, "加载更多失败: " + errorMsg,
Toast.LENGTH_SHORT).show();
        }
        break;
    }
}

// 消息类型常量
private static final int MSG_LOAD_SUCCESS = 1;
private static final int MSG_LOAD_FAILURE = 2;
private static final int TYPE_REFRESH = 1;
private static final int TYPE_LOAD_MORE = 2;

```

9.7.2 创建自定义线程池

```

// 在MainActivity类中添加线程池
private ExecutorService mThreadPool;

/**
 * 初始化线程池
 */
private void initThreadPool() {
    // 创建一个核心线程数为3，最大线程数为6的线程池
    // 非核心线程空闲60秒后会被回收
    mThreadPool = new ThreadPoolExecutor(
        3,                // 核心线程数
        6,                // 最大线程数
        60L,              // 非核心线程空闲超时时间
        TimeUnit.SECONDS, // 超时时间单位
        new LinkedBlockingQueue<Runnable>(), // 工作队列
        Executors.defaultThreadFactory(),    // 线程工厂
        new ThreadPoolExecutor.AbortPolicy() // 拒绝策略
    );
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // 初始化线程池
    initThreadPool();

    // 其他初始化代码...
}

```

9.7.3 使用线程池和Handler加载数据

```
/**
 * 使用线程池和Handler加载更多数据
 */
private void loadMoreDataWithThreadPool(final boolean isRefresh) {
    // 提交任务到线程池
    mThreadPool.execute(new Runnable() {
        @Override
        public void run() {
            try {
                // 生成随机高度
                List<Integer> heights = new ArrayList<>();
                for (int i = 0; i < PAGE_SIZE; i++) {
                    heights.add(mRandom.nextInt(601) + 200);
                }

                // 批量请求图片
                final List<Photo> photos = new ArrayList<>();
                for (int height : heights) {
                    // 同步请求（因为已经在工作线程中）
                    Response<Void> response = mApiService.getPhotoUrl(0, 400,
height, "").execute();
                    if (response.isSuccessful()) {
                        photos.add(new
Photo(response.raw().request().url().toString(), height));
                    } else {
                        throw new IOException("请求失败: " + response.message());
                    }
                }

                // 请求成功，发送成功消息
                Message msg = mMainHandler.obtainMessage(MSG_LOAD_SUCCESS);
                msg.obj = photos;
                msg.arg1 = isRefresh ? TYPE_REFRESH : TYPE_LOAD_MORE;
                mMainHandler.sendMessage(msg);

            } catch (Exception e) {
                // 请求失败，发送失败消息
                Message msg = mMainHandler.obtainMessage(MSG_LOAD_FAILURE);
                msg.obj = e.getMessage();
                msg.arg1 = isRefresh ? TYPE_REFRESH : TYPE_LOAD_MORE;
                mMainHandler.sendMessage(msg);
            }
        }
    });
}

/**
 * 修改下拉刷新监听，使用线程池和Handler
 */
private void setRefreshListener() {
    // 下拉刷新监听
    mRefreshLayout.setOnRefreshListener(new OnRefreshListener() {
        @Override
```

```

        public void onRefresh(RefreshLayout refreshLayout) {
            // 清空旧数据
            mPhotoList.clear();
            // 使用线程池和Handler加载数据
            loadMoreDataWithThreadPool(true);
        }
    });

    // 上滑加载更多监听
    mRefreshLayout.setOnLoadMoreListener(new OnLoadMoreListener() {
        @Override
        public void onLoadMore(RefreshLayout refreshLayout) {
            // 使用线程池和Handler加载更多数据
            loadMoreDataWithThreadPool(false);
        }
    });
}

```

9.7.4 资源释放

```

@Override
protected void onDestroy() {
    super.onDestroy();

    // 清空Handler消息队列，避免内存泄漏
    if (mMainHandler != null) {
        mMainHandler.removeCallbacksAndMessages(null);
    }

    // 关闭线程池，避免资源浪费
    if (mThreadPool != null && !mThreadPool.isShutdown()) {
        mThreadPool.shutdown();
    }
}

```

说明：

- 使用Handler处理子线程到主线程的通信，确保UI更新在主线程执行
- 自定义线程池管理并发任务，避免创建过多线程
- 在onDestroy中释放资源，避免内存泄漏和资源浪费
- 使用消息类型和参数区分不同的操作类型（刷新/加载更多）

这种实现方式比AsyncTask更加灵活和可控，特别适合需要精细控制线程行为的场景。

四、关键功能说明

1. 瀑布流实现

- 使用 `StaggeredGridLayoutManager`（瀑布流布局管理器），设置 `spanCount="2"`（2列）。
- 在 `PhotoAdapter` 的 `onBindViewHolder` 中，设置 `ImageView` 的宽度为 `mItemWidth`（屏幕宽度/列数），高度为随机值（200~800），保证每列宽度一致，高度不同，形成瀑布流效果。

2. 下拉刷新 (SmartRefreshLayout)

- 在 MainActivity 中, 设置 `mRefreshLayout.setOnRefreshListener`, 监听下拉事件。
- 刷新时, 调用 `loadFirstData()` 重新加载首屏数据, 清空旧数据, 用 `mAdapter.setData()` 更新适配器, 最后调用 `mRefreshLayout.finishRefresh()` 停止刷新。

3. 上滑加载更多 (子线程处理)

- 使用 `AsyncTask` (子线程) 执行加载更多逻辑: `doInBackground` 中用 Retrofit 同步请求数据 (因为子线程可以执行同步操作), `onPostExecute` 中更新适配器 (主线程)。
- 加载成功时, 用 `mAdapter.addData()` 添加数据 (更高效), 调用 `mRefreshLayout.finishLoadMore()` 停止加载; 失败时, 调用 `mRefreshLayout.finishLoadMore(false)` 显示错误提示。

4. 图片加载 (Glide+圆角处理)

- 封装 `GlideUtils.loadRoundImage()` 方法, 使用 `RoundedCorners` 变换实现圆角效果 (dp2px 将 dp 转为 px, 避免不同屏幕的适配问题)。
- 设置 `placeholder` (占位图) 和 `error` (错误图), 提升用户体验。

五、测试与调试

- 运行应用, 检查瀑布流是否正确显示 (每列宽度一致, 高度随机)。
- 下拉屏幕, 检查是否触发刷新 (首屏数据重新加载)。
- 上滑到底部, 检查是否触发加载更多 (子线程请求数据, 添加到列表末尾)。
- 断开网络, 检查是否显示错误图 (加载失败时)。
- 查看日志 (Logcat), 如果有错误, 根据 `TAG="MainActivity"` 过滤日志, 定位问题。

六、扩展建议 (可选)

- 增加图片点击事件: 在 `PhotoAdapter` 的 `onBindViewHolder` 中, 给 `holder.ivPhoto` 设置 `OnClickListener`, 实现点击查看大图功能。
- 优化加载更多: 使用 `Paging3` 库 (Jetpack 组件), 更高效地处理分页加载。
- 增加缓存: 使用 `Glide` 的缓存策略 (默认开启), 或者添加 `DiskLruCache` 缓存网络请求数据。
- 自定义刷新头/加载尾: 修改 `SmartRefreshLayout` 的 `refreshHeader` 和 `refreshFooter` 属性, 使用自定义的布局。

七、常见问题解决

- 瀑布流显示异常 (item 宽度不一致):** 检查 `mItemWidth` 的计算是否正确 (屏幕宽度/列数), 并在 `PhotoAdapter` 的 `onBindViewHolder` 中正确设置 `ImageView` 的宽度。
- 下拉刷新不生效:** 检查 `SmartRefreshLayout` 是否正确包裹 `RecyclerView`, 并设置 `enableRefresh="true"` (默认开启)。
- 上滑加载不触发:** 检查 `SmartRefreshLayout` 的 `enableLoadMore="true"` (默认开启), 并确保 `RecyclerView` 的内容足够长 (超过屏幕高度)。

4. **图片加载失败**：检查网络权限是否添加（INTERNET），或者图片地址是否正确（`https://fastly.picsum.photos/id/467/400/205.jpg?hmac=pFewWLKkgu961zCgTP9fHK6TdYTXaRE606fktzdo8bE`）。

八、总结

本项目使用Java语言开发，覆盖了瀑布流界面、下拉刷新、上滑加载、网络请求、图片加载等核心功能。关键亮点包括：

1. **架构清晰**：采用分层设计（model、adapter、network、ui、utils），符合软件工程最佳实践
2. **组件选择**：
 - SmartRefreshLayout实现下拉刷新和上拉加载
 - Retrofit处理网络请求（fastly.picsum.photos API）
 - Glide加载图片并实现圆角效果
 - BaseRecyclerViewAdapterHelper简化适配器开发
3. **性能优化**：
 - 使用AsyncTask在子线程处理上滑加载
 - 适配器使用notifyItemRangeInserted而非notifyDataSetChanged
 - 图片加载使用缓存策略
4. **用户体验**：
 - 瀑布流布局展示不同高度图片
 - 下拉刷新和上拉加载提供流畅交互
 - 图片加载时显示占位图，失败时显示错误图

通过本项目，新手开发者可以学习Android开发的核心概念和实践技巧，掌握常用第三方库的使用方法，为开发更复杂的应用打下坚实基础。

步骤6：SmartRefreshLayout深入优化

目的：深入了解SmartRefreshLayout的高级特性，优化刷新和加载体验。

6.1 SmartRefreshLayout全局配置

目的：在Application中进行全局配置，统一刷新样式。

```
public class MyApplication extends Application {
    @Override
    public void onCreate() {
        super.onCreate();

        // 设置全局默认配置（优先级最低，会被其他设置覆盖）
        SmartRefreshLayout.setDefaultRefreshHeaderCreator(new
        DefaultRefreshHeaderCreator() {
            @NonNull
            @Override
            public RefreshHeader createRefreshHeader(@NonNull Context context,
            @NonNull RefreshLayout layout) {
                // 指定经典刷新头
                return new ClassicsHeader(context)
                    .setDrawableSize(16f) // 设置图标大小
            }
        });
    }
}
```

```

        .set textSizeTitle(14) // 设置标题文字大小
        .set AccentColor(Color.parseColor("#666666")); // 设置强调色
    }
});

// 设置全局默认加载尾
SmartRefreshLayout.setDefaultRefreshFooterCreator(new
DefaultRefreshFooterCreator() {
    @NonNull
    @Override
    public RefreshFooter createRefreshFooter(@NonNull Context context,
@NonNull RefreshLayout layout) {
        // 指定经典加载尾
        return new ClassicsFooter(context)
            .set DrawableSize(16f)
            .set textSizeTitle(14)
            .set AccentColor(Color.parseColor("#666666"));
    }
});
}
}
}

```

6.2 SmartRefreshLayout高级属性配置

目的： 通过XML属性配置SmartRefreshLayout的高级特性。

```

<com.scwang.smart.refresh.layout.SmartRefreshLayout
    android:id="@+id/refresh_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:srlEnableAutoLoadMore="true"           <!-- 是否启用列表惯性滑动到底部时自动加
载更多 -->
    app:srlEnableLoadMoreWhenContentNotFull="false" <!-- 在内容不满一页的时候，是否可
以上拉加载更多 -->
    app:srlEnableOverScrollBounce="true"       <!-- 是否启用越界回弹 -->
    app:srlEnablePreviewInEditMode="true"      <!-- 是否启用编辑模式下预览功能 -->
    app:srlEnablePureScrollMode="false"        <!-- 是否启用纯滚动模式 -->
    app:srlEnableScrollContentWhenLoaded="true" <!-- 是否在加载完成时滚动列表显示新的内
容 -->
    app:srlEnableScrollContentWhenRefreshed="true" <!-- 是否在刷新完成时滚动列表显示新
的内容 -->
    app:srlHeaderHeight="100dp"                <!-- 头部高度 -->
    app:srlHeaderMaxDragRate="2.0"             <!-- 头部最大拖动倍率 -->
    app:srlFooterHeight="60dp"                 <!-- 底部高度 -->
    app:srlFooterMaxDragRate="2.5"             <!-- 底部最大拖动倍率 -->
    app:srlDragRate="0.5">                    <!-- 显示拖动距离/真实拖动距离（默认
0.5，阻尼效果） -->

    <!-- RecyclerView -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</com.scwang.smart.refresh.layout.SmartRefreshLayout>

```

6.3 SmartRefreshLayout监听优化

目的： 优化刷新和加载监听，提供更好的用户体验。

```
// 设置下拉刷新和上滑加载监听
private void setRefreshListener() {
    // 下拉刷新监听
    mRefreshLayout.setOnRefreshListener(new OnRefreshListener() {
        @Override
        public void onRefresh(@NonNull RefreshLayout refreshLayout) {
            // 刷新时，重新加载首屏数据（清空旧数据）
            loadFirstData();

            // 也可以设置刷新超时
            refreshLayout.finishRefresh(2000); // 2秒后自动结束刷新（防止网络问题导致一直刷新）
        }
    });

    // 上滑加载更多监听
    mRefreshLayout.setOnLoadMoreListener(new OnLoadMoreListener() {
        @Override
        public void onLoadMore(@NonNull RefreshLayout refreshLayout) {
            // 加载更多时，用线程池在子线程处理
            loadMoreDataWithThreadPool(false);

            // 也可以设置加载超时
            refreshLayout.finishLoadMore(2000); // 2秒后自动结束加载（防止网络问题导致一直加载）
        }
    });

    // 添加刷新状态变化监听器（可选）
    mRefreshLayout.setOnMultiListener(new SimpleMultiListener() {
        @Override
        public void onStateChanged(@NonNull RefreshLayout refreshLayout,
                                   @NonNull RefreshState oldState,
                                   @NonNull RefreshState newState) {
            // 状态变化监听，可以根据不同状态执行不同操作
            if (newState == RefreshState.Refreshing) {
                // 正在刷新
                Log.d(TAG, "正在刷新...");
            } else if (newState == RefreshState.Loading) {
                // 正在加载
                Log.d(TAG, "正在加载更多...");
            }
        }
    });

    @Override
    public void onHeaderMoving(RefreshHeader header, boolean isDragging,
                               float percent,
                               int offset, int headerHeight, int
                               maxDragHeight) {
        // 下拉头部移动（可实现视差效果）
        // percent: 下拉的百分比 0.0f-2.0f
    }
}
```

```

        // offset: 下拉的像素偏移量
    }
});
}

```

6.4 SmartRefreshLayout状态处理

目的： 处理不同的刷新和加载状态，提供更好的用户反馈。

```

// 成功加载数据后调用
private void handleLoadSuccess(List<Photo> photos, boolean isRefresh) {
    if (isRefresh) {
        // 下拉刷新成功
        mAdapter.setList(photos);
        mRefreshLayout.finishRefresh(true); // 传入true表示刷新成功
        // 显示刷新成功提示
        Toast.makeText(this, "刷新成功", Toast.LENGTH_SHORT).show();
    } else {
        // 上滑加载更多成功
        if (photos.size() > 0) {
            mAdapter.addData(photos);
            mRefreshLayout.finishLoadMore(true); // 传入true表示加载成功
        } else {
            // 没有更多数据了
            mRefreshLayout.finishLoadMoreWithNoMoreData(); // 完成加载并标记没有更多
数据
        }
    }
}

// 加载数据失败后调用
private void handleLoadFailure(String errorMsg, boolean isRefresh) {
    if (isRefresh) {
        // 下拉刷新失败
        mRefreshLayout.finishRefresh(false); // 传入false表示刷新失败
    } else {
        // 上滑加载更多失败
        mRefreshLayout.finishLoadMore(false); // 传入false表示加载失败
    }
    // 显示错误提示
    Toast.makeText(this, errorMsg, Toast.LENGTH_SHORT).show();
}

```

6.5 自定义刷新头部

目的： 创建个性化的刷新头部，提升品牌辨识度和用户体验。

```

/**
 * 自定义刷新头部
 */
public class CustomRefreshHeader extends SimpleComponent implements RefreshHeader
{
    private TextView mTitleView;
    private ImageView mProgressView;
    private Animation mRotateAnimation;
}

```

```

public CustomRefreshHeader(Context context) {
    super(context);
    initView(context);
}

private void initView(Context context) {
    // 加载布局文件
    LayoutInflater.from(context).inflate(R.layout.custom_refresh_header,
this);

    // 初始化控件
    mTitleView = findViewById(R.id.tv_title);
    mProgressView = findViewById(R.id.iv_progress);

    // 初始化动画
    mRotateAnimation = new RotateAnimation(0, 360,
        Animation.RELATIVE_TO_SELF, 0.5f,
        Animation.RELATIVE_TO_SELF, 0.5f);
    mRotateAnimation.setDuration(800);
    mRotateAnimation.setRepeatCount(Animation.INFINITE);
    mRotateAnimation.setInterpolator(new LinearInterpolator());
}

@Override
public void onStateChanged(@NonNull RefreshLayout refreshLayout,
    @NonNull RefreshState oldState,
    @NonNull RefreshState newState) {
    switch (newState) {
        case PullDownToRefresh: // 下拉刷新
            mTitleView.setText("下拉刷新");
            mProgressView.clearAnimation();
            break;
        case ReleaseToRefresh: // 释放刷新
            mTitleView.setText("释放立即刷新");
            mProgressView.clearAnimation();
            break;
        case Refreshing: // 刷新中
            mTitleView.setText("正在刷新...");
            mProgressView.startAnimation(mRotateAnimation);
            break;
        case RefreshFinish: // 刷新完成
            mTitleView.setText("刷新完成");
            mProgressView.clearAnimation();
            break;
    }
}

@Override
public int onFinish(@NonNull RefreshLayout layout, boolean success) {
    // 刷新完成
    mProgressView.clearAnimation();
    if (success) {
        mTitleView.setText("刷新成功");
    } else {
        mTitleView.setText("刷新失败");
    }
}

```

```
    }  
    return 500; // 延迟500毫秒关闭  
}  
  
// 其他必要的接口实现...  
}
```

6.6 SmartRefreshLayout进阶用法

目的：展示SmartRefreshLayout的进阶功能，满足复杂业务需求。

6.6.1 自动刷新和加载

```
// 自动刷新（不需要用户下拉）  
mRefreshLayout.autoRefresh();  
  
// 自动刷新（延迟300毫秒）  
mRefreshLayout.autoRefresh(300);  
  
// 自动加载更多（不需要用户上拉）  
mRefreshLayout.autoLoadMore();  
  
// 自动加载更多（延迟300毫秒）  
mRefreshLayout.autoLoadMore(300);
```

6.6.2 动态启用/禁用刷新和加载

```
// 启用/禁用下拉刷新  
mRefreshLayout.setEnableRefresh(true); // 启用  
mRefreshLayout.setEnableRefresh(false); // 禁用  
  
// 启用/禁用上拉加载  
mRefreshLayout.setEnableLoadMore(true); // 启用  
mRefreshLayout.setEnableLoadMore(false); // 禁用  
  
// 设置是否在没有更多数据时启用上拉加载（false表示没有更多数据时不显示上拉加载）  
mRefreshLayout.setEnableLoadMoreWhenContentNotFull(false);  
  
// 设置是否在刷新头部和加载尾部处理滚动冲突（默认true）  
mRefreshLayout.setEnableNestedScroll(true);
```

6.6.3 无更多数据和重置状态

```
// 完成加载并标记没有更多数据  
mRefreshLayout.finishLoadMoreWithNoMoreData();  
  
// 恢复没有更多数据的原始状态  
mRefreshLayout.setNoMoreData(false);  
  
// 重置所有状态  
mRefreshLayout.resetNoMoreData(); // 重置没有更多数据状态
```

6.6.4 获取当前状态

```
// 获取当前刷新状态
RefreshState state = mRefreshLayout.getState();

// 判断是否正在刷新
boolean isRefreshing = state == RefreshState.Refreshing;

// 判断是否正在加载更多
boolean isLoading = state == RefreshState.Loading;

// 判断是否处于没有更多数据状态
boolean isNoMoreData = mRefreshLayout.isNoMoreData();
```

6.7 SmartRefreshLayout与RecyclerView的优化配合

目的： 优化SmartRefreshLayout与RecyclerView的协作，提高性能和用户体验。

```
// 设置滚动监听，实现上滑加载更多的预加载功能
mRecyclerView.addOnScrollListener(new RecyclerView.OnScrollListener() {
    @Override
    public void onScrolled(@NonNull RecyclerView recyclerView, int dx, int dy) {
        super.onScrolled(recyclerView, dx, dy);

        // 获取布局管理器
        StaggeredGridLayoutManager layoutManager =
            (StaggeredGridLayoutManager) recyclerView.getLayoutManager();

        // 获取最后一个可见项位置
        int[] lastVisibleItemPositions = new int[2]; // 2列瀑布流
        layoutManager.findLastVisibleItemPositions(lastVisibleItemPositions);
        int lastVisibleItemPosition = Math.max(lastVisibleItemPositions[0],
            lastVisibleItemPositions[1]);

        // 获取总项数
        int totalItemCount = layoutManager.getItemCount();

        // 预加载：当最后可见项距离末尾小于5个时，触发加载更多
        if (totalItemCount - lastVisibleItemPosition < 5 && dy > 0) {
            // 如果不是正在加载，且不处于没有更多数据状态，则触发加载更多
            if (!mRefreshLayout.isLoading() && !mRefreshLayout.isNoMoreData()) {
                mRefreshLayout.autoLoadMore();
            }
        }
    }
});

// 优化：避免刷新和加载同时进行
mRefreshLayout.setDisableContentWhenRefresh(true); // 刷新时禁用内容
mRefreshLayout.setDisableContentWhenLoading(true); // 加载时禁用内容

// 优化：设置刷新和加载的触发阈值
mRefreshLayout.setHeaderTriggerRate(1.0f); // 刷新触发阈值（默认1.0，表示露出100%的Header时触发）
```

```
mRefreshLayout.setFooterTriggerRate(1.0f); // 加载触发阈值（默认1.0，表示露出100%的Footer时触发）
```

```
// 优化：设置刷新和加载的最大时长（超时自动结束）
```

```
mRefreshLayout.setRefreshTimeout(10000); // 刷新最大时长10秒
```

```
mRefreshLayout.setLoadMoreTimeout(10000); // 加载最大时长10秒
```